

Chosen Is Not Paid

Email and password are now the main way in, phone sign-in keeps working for every account that already exists, and signup asks which programme somebody wants before it asks for anything else. The hard part was not the forms. It was keeping four things apart that one column would have merged: the programme **chosen**, the subscription's **state**, the **payment**, and what somebody is **entitled** to right now. No payment provider exists, so nothing can be entitled to anything — and the code makes that structurally true rather than merely currently true.

Branch **claude/create-app-repository-1cxsih**

Code at **7feadac**

Unit tests **736 passed, 20 skipped**

Browser tests **54 passed**

Mutations **19 / 19 caught**

Ledger touched **none**

Email provider **still none**

READ THIS FIRST: THREE THINGS THIS MILESTONE DOES NOT DO

No email is delivered anywhere. No provider is chosen, so `ConsoleSender` is what runs and it writes what it would have sent to the log. Verification and reset codes are exercised end to end in development, CI and preview by reading that log. They reach no inbox. Section H says exactly what remains.

No Entrepreneur subscription can become active. Choosing the paid programme writes a `pending_payment` row that grants nothing. There is deliberately no `activate()` function anywhere: with no payment provider implemented, anything able to set `active` today would set it because a button was pressed.

No existing account was migrated, backfilled or deleted. Migration 0016 is additive only. Phone accounts keep their invented `@phone.afrinext.local` address exactly as it is, keep their sessions, their roles, their wallet and their ledger history. Section E is the evidence.

- A · The four things kept apart

B · What was built

C · Codes, and what they cannot leak
- D · Reset, logout, sessions

E · Existing accounts

F · The screens

A

The four things kept apart

Your constraint was precise: *"choisir Entrepreneur ne veut jamais dire que l'abonnement est payé ou actif"*, and *"ne crée pas une architecture qui empêcherait un utilisateur Vendeur de devenir Entrepreneur sans créer un deuxième compte"*. Both are structural, so both are enforced by where the data lives rather than by a rule somebody has to remember.

THING	WHERE IT LIVES	WHAT IT MEANS
Programme chosen	users.programme	A declared intent. Grants no role, unlocks no feature, and is never consulted by <code>authorize()</code> .
Subscription state	programme_subscriptions.status	<code>pending_payment</code> → <code>active</code> → <code>past_due</code> / <code>cancelled</code> / <code>expired</code> . Today only the first is reachable.
Payment	orders / payments	Not touched by this milestone at all. <code>iPayMoney</code> remains unimplemented.
Entitlement	Derived, never stored	<code>isEntitled()</code> — an <code>active</code> row whose period covers the clock. A cached entitlement is a bill somebody stopped paying that nobody noticed.

Why a column on `users` and not a row somewhere else

Because a person has exactly one current programme, and because moving between them has to be an `UPDATE` on the account they already have. **Same `users.id` throughout** — which is what their roles, their store, their wallet, their orders and every ledger entry ever posted against them are keyed on. There is no path in the code that asks anybody to open a second account, and the test suite asserts there is not: after an upgrade, `select count(*) from users` is still 1 and `auth_user_id` is unchanged.

Why there is no `activate()`

THE ABSENCE IS THE FEATURE

`active` is declared in the state machine and is **unreachable from application code**. Writing an activation function now would mean either taking a payment reference nothing can verify, or taking none at all — and the second is precisely how a subscription becomes active because a button was pressed.

A test asserts the absence rather than describing it: after choosing Entrepreneur, `select count(*) from programme_subscriptions where status = 'active' is 0`, and the browser journey additionally asserts **no order, no payment and no ledger account** was created. If a later change adds a way to activate without a verified payment, that test fails.

The price is data

2 000 FCFA is what the business decided today, not a constant of the software. It is a default in `PROGRAMME_PRICES` and an `UPDATE` in `platform_settings` under `programme.pricing` — the same posture as the OTP policy. Each field is validated on its own, so a malformed row cannot make the programme free or land it in the wrong currency; anything unusable falls back to the reviewed default rather than through it.

XOF has zero decimal places, so 2 000 francs is `2000n` minor units. Writing `200000n` — the reflex from two-decimal currencies — would charge a hundred times the price and nothing downstream would catch it. That is mutation E16, and it is caught.

B

What was built

Migration 0016 — additive only

```
-- one CHECK widened, one column added, one table created. Nothing rewritten.
ALTER TABLE otp_challenges DROP CONSTRAINT IF EXISTS otp_purpose_valid;
ALTER TABLE otp_challenges ADD CONSTRAINT otp_purpose_valid CHECK (purpose IN
    ('sign_in','verify_identity','step_up','email_verification','password_reset'));

ALTER TABLE users ADD COLUMN IF NOT EXISTS programme TEXT NOT NULL DEFAULT 'vendeur';

CREATE TABLE programme_subscriptions (... status, price_minor, currency, period ...);
CREATE UNIQUE INDEX programme_subscriptions_one_live ON programme_subscriptions
    (user_id, programme) WHERE status IN ('pending_payment','active','past_due');
```

Four rules are enforced by PostgreSQL rather than by application code, because the application is not the only thing that will ever write to these tables:

CONSTRAINT	REFUSES
programme_subscriptions_one_live	A second live subscription for one person. Two <code>pending_payment</code> rows is two invoices for one month, and whichever the provider confirms leaves the

CONSTRAINT	REFUSES
	other looking unpaid for ever. Cancelled and expired rows are excluded, so history is never overwritten.
..._active_has_period	An <code>active</code> subscription with no billing period.
..._price_positive	A free or negative price on a paid programme.
users_programme_valid	A programme the application does not know.

Files

LAYER	FILE	WHAT IT DOES
Migration	db/migrations/0016_email_password_auth.sql	The above.
Schema	db/src/schema/identity.ts	<code>users.programme</code> , <code>programme_subscriptions</code> , widened purpose CHECK. Kept in step so CI's rebuild-from-zero still matches.
Domain	core/src/auth/email-identity.ts	Verification and reset, on the existing <code>otp_challenges</code> store. Synthetic-address quarantine.
Domain	core/src/programme/index.ts	Programme choice, subscription lifecycle, entitlement, pricing from settings.
Limits	core/src/ratelimit/index.ts	<code>perEmailPerHour</code> on the stored policy, <code>emailSendRules()</code> .
Routes	web/app/api/v1/auth/*	verify (POST/PUT), password/forgot, password/reset, programme (GET/POST).
Screens	sign-up · sign-in · password-reset · verify-email · programme	Section F.

What was not touched, checked rather than claimed

The whole diff is **50 files, +4 871 / −26**. Nine of those files are one line each: the existing browser tests that sign in by phone now tap through to that flow first. The 26 deletions are all inside files this milestone rewrote.

AREA	FILES CHANGED BY THIS MILESTONE
Ledger, commissions, refunds, payments, iPayMoney	None. <code>git diff --name-only</code> over the milestone matches nothing under those modules except <code>e2e/refunds.spec.ts</code> , whose entire change is the one-line phone tap above.

AREA	FILES CHANGED BY THIS MILESTONE
Design tokens and non-auth screens	None. <code>tokens.css</code> is untouched; the only non-auth UI file in the diff is the locale layout, which gained the banner and the sign-out item.
<code>packages/*</code> importing Next.js	None , still enforced by the eslint rule rather than by discipline.

REFERRAL AND THE ACTIVATION BONUS, PRECISELY

No referral model was invented and no referral code was written. Searching the repository for the word does return hits, and honesty is better than a clean-sounding claim: they are all pre-existing, from Phase 0 and Phase 1, and none is logic. A `referral_terms` value in the legal-document kind CHECK, a `referral` value in the fee-line key column, and two comments in `ledger` and `money/allocate` explaining why a commission basis is not always the gross.

For the activation bonus the claim is absolute and was verified the same way: **there is no constant, no column, no table, no transaction, no state and no code path named for it or implementing it — no REFERRAL_ACTIVATION , no 1 000 FCFA anywhere.** Nothing was removed either, because nothing was ever there.

The programme endpoint says what it means

```
{ "data": { "programme": "entrepreneur",
  "subscriptionStatus": "pending_payment",
  // named `entitled`, never `active`, and never derived from the choice
  "entitled": false,
  "priceMinor": "2000", "currency": "XOF" } }
```

Minor units go out as a **string** with the currency alongside, because `JSON.stringify` throws on a bigint — a useful accident that forces every transport to say how an amount is represented rather than letting one quietly become a float.

c Codes, and what they cannot leak

Email verification and password reset use **the authoritative challenge mechanism**, not a second one. Same table, same keyed HMAC, same database-enforced properties. Two values were added to the `purpose` CHECK and nothing else changed.

PROPERTY	ENFORCED BY
The code exists in exactly one place, as a hash	<code>otp_challenges.code_hash</code> , HMAC-SHA256 under a key derived from the application secret. A plain digest of a six-digit code is reversible by trying all million inputs; the key is what a stolen table does not contain.
One live challenge per identifier and purpose	Partial unique index. Banking several usable codes and spending each one's attempt budget is not possible.
Bounded attempts	<code>where attempts < max_attempts</code> — a predicate, not a number this process is trusted to remember.
Single use	<code>where consumed_at is null</code> .
Verification and reset cannot be swapped	The purpose is in the hash and in the lookup. A verification code presented to the reset endpoint is refused, and the reverse. Mutation E7.

No leak about whether an address exists

You asked for this explicitly. It holds in three places at once, and the third is the one that is usually forgotten:

1. **The body and the status are identical.** An address with an account, one without, and a phone account's unreachable synthetic address all answer `{ "sent": true }` .
2. **Every verification failure is identical.** Wrong code, expired code, no challenge at all, exhausted attempts, synthetic address — one `400 auth.otp_invalid` . The real reason goes to the log.
3. **The rate limit is consumed BEFORE the address is judged.** Limiting only real, reachable addresses would make the limiter itself the oracle: an attacker learns which addresses exist by seeing which ones eventually answer 429 and which ones never do. Every request costs a token, including the ones that lead nowhere. Mutation E11.

TESTED, NOT ASSERTED

The browser test drives the real form twice — once with an address that has an account, once with one that does not — and compares **the rendered text of the notice** and the step the form advances to. Same words, same next screen. A form that only advanced for real addresses would leak exactly what the message refuses to.

Issuance is limited, and the limits are configuration

Bounding attempts alone protects nothing if fresh codes can be requested without limit. The first draft of this carried `limit: 5` inline — which is the defect review decision

7 already ruled on for SMS — so the rules moved into `ratelimit` as `emailSendRules()`, reading a new `perEmailPerHour` field on the stored policy.

Changing a limit stays an `UPDATE`, not a deploy.

It is a separate field from `perPhonePerHour` because the two cost different things: an SMS is money out of the door on every send, an email spends the sending domain's reputation, and a mailbox that receives forty Afrinext codes in an hour marks the next one as spam for everybody.

The synthetic address, and why it gets its own helper

Phone signup mints `<digits>@phone.afrinext.local` to satisfy Better Auth's `NOT NULL UNIQUE` email column. That address is a placeholder wearing the costume of a real one: valid to the schema, unique, and it will never receive mail. **The TLD does not resolve, and anybody may stand up a host that answers for it.**

One helper, three call sites — sign-in refuses one, reset never sends to one, the banner offers "add an email" rather than "verify" one. Spelling the check inline in three places is how the fourth place forgets.

D

Reset, logout, sessions

A reset ends every session, and that is the point of it

Not housekeeping afterwards. **A reset that leaves an attacker's session alive has achieved nothing:** the person whose account it is changes their password, feels safe, and the intruder is still signed in on another device. Better Auth's `session` table is the only session store there is, so deleting the rows *is* signing those devices out — one statement, inside the same transaction as the password write.

The current session goes too. The screen then sends the person to sign in with the password they just chose, which is the honest ending: a reset means every existing session is suspect, including this one.

PROVED IN A BROWSER, WITH TWO OF THEM

The test signs the same account in on a second browser context, resets the password from the first, and then asserts that the second one's `/api/v1/me` answers **401** and that `select count(*) from session` for that account is **0**. The old password is then refused and the new one works. Mutation E8 deletes the revocation; it is caught.

Logout

Sign-out is in the Menu drawer and goes through Better Auth, which **deletes the session row**. The token is dead for anybody holding it the moment it returns, not merely missing from this browser — a client that only cleared its own cookie would leave a live row behind. The browser test asserts the row count, then that `/api/v1/me` answers 401, then that `/fr/wallet` redirects to sign-in.

A second test asserts the **opposite** for the other devices: signing out on one does *not* sign out the others. Signing out is not a security event on the account; a reset is. That line is easy to blur in the wrong direction.

Better Auth writes the credential, and Afrinext writes it back

The reset writes `account.password` directly, which is the one place this milestone reaches into Better Auth's tables. It is safe because Better Auth is configured with **Afrinext's own script** on both sides — but that is a configuration fact, and a configuration fact deserves a test. So one drives the real Better Auth instance: reset the password through the domain, then sign in through `signInEmail` with the new one and fail with the old.

The update is scoped to `providerId = 'credential'`, because an account can hold one row per provider. Unscoped, the write puts a password hash into a field an OAuth provider owns, and on an account with several rows it becomes luck rather than logic which one Better Auth then verifies against. Mutation E10 removes the scope — and survived the first matrix, because every account in the suite had exactly one row for it to be wrong about. Section G.

E

Accounts that existed before this milestone

Your instruction was to preserve them without a mass backfill and without turning their invented addresses into real ones. Nothing migrates them. The evidence is a test group of its own, driving the real phone flow:

CLAIM	HOW IT IS PROVED
Phone sign-in still works	The whole existing browser suite signs in by phone — 46 tests, all passing. The screen now opens on email, so each taps through to the phone flow first.
The synthetic address is untouched	Asserted to still end in <code>@phone.afrinext.local</code> and to be unreachable. Not rewritten, not deleted.

CLAIM	HOW IT IS PROVED
Existing sessions survive	Session count is asserted before and after. Nothing in this change revokes them — revoking every existing seller's session for a schema addition would be an outage dressed as caution.
Roles and permissions hold	Asserted across a programme change in both directions.
The programme default is the migration's, not a backfill	<code>vendeur</code> , with no subscription row at all.
They can take the paid programme without a second account	Same <code>users.id</code> , same <code>auth_user_id</code> , same session, one row in <code>users</code> .
They cannot be taken over through the reset flow	The interesting attack, below.

THE ATTACK THE QUARANTINE EXISTS TO STOP

A phone account **has** an address in `user.email` . A reset flow that merely looked the address up would send a code to `<digits>@phone.afrinext.local` — a domain that does not exist, which anybody could register and stand up a mail server for. Whoever did that would then hold a password-reset code for a real seller's account, with its store, its wallet and its ledger balance.

Two layers stop it: nothing issues a code to such an address, and nothing accepts one. **The second layer is what my first test failed to check** — see section G.

Nothing was removed

Per your instruction not to remove the old mechanism until compatibility is assured: **the phone OTP flow is unchanged**. Not deprecated, not hidden, not behind a flag. It is one tap from the sign-in screen and labelled as a choice rather than as a fallback, because for those accounts it is not an alternative — it is the only way in, and they have no password at all.

F The screens

Nine screenshots from the running production build at 390 px, the phone this is designed for. **The Sahel design is unchanged** — no token moved, no other screen was touched. What is new here uses the palette that was already validated.

Créer un compte

Choisissez votre programme

Vous pourrez changer de programme plus tard, sans créer de deuxième compte.

☒ **Afrinext Vendeur** Gratuit

Ouvrez une boutique et vendez sur la place de marché.

☐ **Afrinext Entrepreneur** 2 000 XOF / mois

Le programme complet, avec l'accompagnement Afrinext.

Continuer

Vous avez déjà un compte ? [Se connecter](#)

Accueil

Explorer

Vendre

Bibliothèque

Menu

Signup opens on the programme, before it asks for anything else. The free one is the default because most people are it.

Créer un compte

Choisissez votre programme

Vous pourrez changer de programme plus tard, sans créer de deuxième compte.

☐ **Afrinext Vendeur** Gratuit

Ouvrez une boutique et vendez sur la place de marché.

☒ **Afrinext Entrepreneur** 2 000 XOF / mois

Le programme complet, avec l'accompagnement Afrinext.

Aucun paiement n'a été effectué. Votre abonnement est en attente de paiement. Le paiement en ligne n'est pas encore disponible. Nous vous préviendrons dès qu'il le sera.

Continuer

Vous avez déjà un compte ? [Se connecter](#)

Accueil

Explorer

Vendre

Bibliothèque

Menu

Choosing the paid programme. **The sentence saying nothing has been charged is on screen before the choice is submitted**, not in a confirmation afterwards — by which point somebody already believes they have bought something.

Créer un compte

Votre nom

Aïcha Abdou

Adresse e-mail

aïcha.1788394910653@example.com

Mot de passe

.....

Au moins 10 caractères.

Créer mon compte

Retour

Vous avez déjà un compte ? [Se connecter](#)

Accueil

Explorer

Vendre

Bibliothèque

Menu

Name, address, password. The ten-character minimum is stated and also enforced server-side, twice.

Votre adresse
aicha.1788394910653@example.com
n'est pas encore vérifiée.

[Vérifier](#) [Plus tard](#)

Bienvenue sur

Afrinext.

 EN

VOTRE ACTIVITÉ AFRINEXT



Solde disponible

0 XOF

Solde en attente 0 XOF

Voir le portefeuille →

SERVICES AFRINEXT

 Explorer

 Formations

 Bibliothèque

 Commandes

 Portefeuille

 Vendre

Catégories

Six façons de vendre sur Afrinext.

 Accueil

 Explorer

 Vendre

 Bibliothèque

 Menu

Créer un compte

Avant de continuer

Pour créer votre compte Afrinext, vous devez accepter les documents suivants dans leur version en vigueur.

Politique de confidentialité
v0.0.0-placeholder · Langue: fr

Conditions d'utilisation
v0.0.0-placeholder · Langue: fr

☐ L'acceptation est obligatoire pour créer et utiliser votre compte.

J'accepte ces documents

 Accueil

 Explorer

 Vendre

 Bibliothèque

 Menu

The consent gate, unchanged. The account is `pending_consent` and resolves to no actor until this is accepted — the same door email sign-up meets as the phone flow.

Votre adresse
aicha.1788394910653@example.com
n'est pas encore vérifiée.

[Vérifier](#) [Plus tard](#)

Bienvenue sur

Afrinext.

 EN

VOTRE ACTIVITÉ AFRINEXT



Solde disponible

0 XOF

Solde en attente 0 XOF

Voir le portefeuille →

SERVICES AFRINEXT

 Explorer

 Formations

 Bibliothèque

 Commandes

 Portefeuille

 Vendre

Catégories

Six façons de vendre sur Afrinext.

 Accueil

 Explorer

 Vendre

 Bibliothèque

 Menu

The dashboard, reachable with the address unverified. The banner offers verification and can be dismissed; it gates nothing, and there is no interstitial.

Votre adresse
aicha.1788394910653@example.com
n'est pas encore vérifiée.

[Vérifier](#) [Plus tard](#)

Vérifiez votre adresse e-mail

Nous avons envoyé un code à
aicha.1788394910653@example.com. Saisissez-le ci-dessous.

Code de vérification

0 0 0 0 0 0

Valider mon adresse

Renvoyer le code

Vous pouvez utiliser Afrinext sans vérifier tout de suite.

Plus tard

 Accueil

 Explorer

 Vendre

 Bibliothèque

 Menu

The verification screen, with a real exit. “Plus tard” is a link back to the app, not a nag that reopens.

Se connecter

Adresse e-mail

Mot de passe

[Mot de passe oublié ?](#)

Se connecter

Utiliser mon numéro de téléphone

Vous n'avez pas encore de compte ? [Créer mon compte](#)

 Accueil

 Explorer

 Vendre

 Bibliothèque

 Menu

Réinitialiser le mot de passe

Indiquez votre adresse e-mail. Si un compte y est associé, vous recevrez un code.

Adresse e-mail

Envoyer le code

 Accueil

 Explorer

 Vendre

 Bibliothèque

 Menu

Votre adresse
aicha.1788394910653@example.com
n'est pas encore vérifiée.

[Vérifier](#) [Plus tard](#)

COMPTE

Choisissez votre programme

Programme choisi: En attente de paiement

Choisissez votre programme

Vous pourrez changer de programme plus tard, sans créer de deuxième compte.

☐ Afrinext Vendeur Gratuit

Ouvrez une boutique et vendez sur la place de marché.

☒ Afrinext Entrepreneur 2000 XOF / mois

Le programme complet, avec l'accompagnement Afrinext.

Aucun paiement n'a été effectué. Votre abonnement est en attente de paiement. Le paiement en ligne n'est pas encore disponible. Nous vous préviendrons dès qu'il le sera.

Changer de programme

 Accueil

 Explorer

 Vendre

 Bibliothèque

 Menu

Sign-in: email and password first, the phone flow one tap below it, and “mot de passe oublié” where a person looks for it.

Forgotten password. The next step is the same whether or not the address has an account.

Changing programme after signup. The status shown is the subscription's, not the programme's: “En attente de paiement” is the truth, and “Entrepreneur” alone would read as a subscription that is running.

G

Tests, mutations, and six defects found before you saw them

Results

SUITE	RESULT	WHAT IT COVERS
packages/core (whole suite)	736 passed, 20 skipped	Everything. Run one at a time — see the note below.
auth/email-identity.test.ts	28	Verification, reset, the quarantine, issuance limits, the enumeration surface.
auth/email-password.test.ts	16	The real Better Auth instance: signup, sign-in, duplicates, sign-out, existing phone accounts.
programme/programme.test.ts	20	Choice, upgrade in place, entitlement, the four database constraints, pricing from settings.
e2e/email-auth.spec.ts	8	The journeys in a real browser against a production build.
e2e (whole suite)	54	Including the 46 that already existed.
packages/il8n	14	49 new keys in fr and en; parity, placeholders, plurals.
lint / typecheck / build	clean	--max-warnings 0 across all six packages.

IF YOU RE-RUN THESE

The suite truncates the tables it touches, so **two vitest runs against the same TEST_DATABASE_URL at once deadlock each other** and fail in `ledger/concurrency`, `content/storage-equivalence` and `payments/ipaymoney-replay` — modules with nothing to do with this work. I did that to myself once and spent a few minutes believing I had broken the ledger. Run one at a time. Similarly, `pnpm test:e2e` serves whatever `.next` already holds, so a source change needs `pnpm build` first; CI does that as its own step.

Mutation matrix

All 19 mutations are caught. Every one fails at least one test, so each guarantee in this document is checked by something that would break if the guarantee were removed — rather than merely described in prose.

Each mutation is applied to the source in a detached worktree with its own database, **the whole core suite is run**, and the file is restored. Not a subset: narrowing the suite is how a mutation survives for the wrong reason, which the Phase 5 matrix learned the hard way.

#	MUTATION – WHAT IT BREAKS	RESULT	KILLED BY
1	E1-code-sent-to-synthetic A verification or reset code IS issued and sent to a phone account's invented <code>@phone.afrinext.local</code> address.	CAUGHT	email verification > sends nothing to the synthetic address phone signup invents +2 more
2	E2-reset-through-synthetic <code>resetPassword</code> stops checking the address, so a code presented for a synthetic address is honoured. The second layer of the quarantine.	CAUGHT	password reset > refuses a synthetic address even when a live code exists for it
3	E3-verify-synthetic <code>confirmEmailVerification</code> stops checking, so a mailbox that cannot exist can be marked verified.	CAUGHT	email verification > refuses to verify a synthetic address even with a code in hand
4	E4-synthetic-match-loosened <code>endsWith</code> becomes <code>includes</code> : a real address merely containing the string is quarantined, and a subdomain slips past.	CAUGHT	synthetic addresses > does not mistake a real address that merely contains the domain
5	E5-any-code-verifies The challenge result is ignored — any code, or none, verifies an address.	CAUGHT	email verification > refuses a wrong code, an expired one and a reused one identically +3 more
6	E6-any-code-resets The challenge result is ignored — any code resets a password.	CAUGHT	password reset > refuses a wrong code and leaves the old password working +2 more
7	E7-purpose-not-separated The reset path looks up an <code>email_verification</code> challenge, so a verification code resets a password.	CAUGHT	password reset > sets a new password and revokes every session +3 more
8	E8-sessions-survive-reset A reset no longer deletes the account's sessions: the intruder stays signed in.	CAUGHT	a password reset ends every session > signs out the other device, and the current one +1 more

#	MUTATION – WHAT IT BREAKS	RESULT	KILLED BY
9	E9-weak-password-accepted The minimum length check is removed from the reset path.	CAUGHT	password reset > refuses a password shorter than the minimum without spending the code
10	E10-wrong-account-row The password write is no longer scoped to <code>providerId = 'credential'</code> .	CAUGHT	password reset > writes only the credential row, not every provider the account has
11	E11-limit-after-existence-check The rate limit is consumed only for reachable addresses, turning the limiter into an account-existence oracle.	CAUGHT	issuance rate limits > refuses once the hourly allowance for an address is spent+3 more
12	E12-no-issuance-limit The per-address hourly limit and the resend cooldown are removed from <code>emailSendRules</code> .	CAUGHT	issuance rate limits > refuses once the hourly allowance for an address is spent+2 more
13	E13-choice-activates Choosing the paid programme writes <code>active</code> with a running period — a subscription because a button was pressed.	CAUGHT	accounts that existed before this milestone > can take the entrepreneur programme without a second account 2+3 more
14	E14-entitled-from-choice <code>isEntitled</code> accepts anything that is not cancelled, so <code>pending_payment</code> grants the programme.	CAUGHT	entitlement > is false for pending payment, and for an active row whose period has run out
15	E15-period-ignored <code>isEntitled</code> stops consulting the billing period, so an expired subscription nobody swept still counts.	CAUGHT	entitlement > is false for pending payment, and for an active row whose period has run out
16	E16-price-hundredths The price becomes <code>200000n</code> minor units — the two-decimal reflex, and a hundred times the price in a zero-decimal currency.	CAUGHT	programme choice > freezes the price in XOF minor units, which have no decimals+2 more
17	E17-malformed-setting-widens <code>loadProgrammePrice</code> stops validating field by field, so a malformed settings row can make the programme free or set a bogus currency.	CAUGHT	pricing is data > refuses field by field, so a malformed row cannot make the programme free
18	E18-second-live-subscription <code>chooseProgramme</code> stops finding the existing live row, so a repeated choice tries to write a second invoice.	CAUGHT	programme choice > is idempotent: a double submit does not create a second subscription+2 more
19	E19-downgrade-leaves-invoice Going back to the free programme leaves the unpaid subscription live — an invoice nobody	CAUGHT	moving between programmes > cancels an unpaid subscription when somebody goes back to the free programme 198

#	MUTATION – WHAT IT BREAKS	RESULT	KILLED BY
	agreed to, and a slot the unique index will not release.		

Three defects that were in the code before the tests found them

1 · SIGNUP NEVER SIGNED ANYBODY IN

`emailAndPassword.autoSignIn` is `false`, deliberately — consent has not been given at that point, so a session issued by signup would be a session for an account that resolves to no actor. My flow assumed a session existed after `signUp.email` and called the programme endpoint next. **That call would have answered 401**, and so would the consent fetch after it. The screen now signs in explicitly. Found by writing the integration test against the real Better Auth instance instead of against my reading of its configuration.

2 · A DUPLICATE ADDRESS RETURNS SUCCESS

`signUpEmail` on an address that already exists returns a *success* shape — a user object with `token: null` — and creates nothing. A caller reading "no error" as "account created" sails straight past it. The screen now tells the two apart the only way available from outside: it tries the credentials, and reports the address as taken when they do not work. A signup form saying an address is taken IS an enumeration surface, and an unavoidable one — a form that silently accepted a duplicate would be broken instead. The endpoints that **do** have a choice, reset and verification, are the ones that answer identically.

3 · APP_URL DECIDES WHETHER EMAIL SIGN-IN WORKS AT ALL

It becomes Better Auth's `baseURL`, and Better Auth **refuses any request whose Origin does not match it** — `INVALID_ORIGIN`, before the handler runs. The browser servers answered on `127.0.0.1:31xx` while claiming to be `localhost:3000`, so every signup and every email sign-in was rejected.

The same misconfiguration in production refuses every email sign-in while the phone flow keeps working, because the phone flow goes through Afrinext's own plugin endpoints and never meets that check — which is exactly what makes the breakage look partial and impossible to diagnose. `render.yaml` already declared `APP_URL` as `sync: false`; its comment said a wrong value meant "links point at localhost". It now says what it actually costs. **This is item 1 in section H.**

And three weak tests of my own

The matrix is worth running because it grades the tests, not the code. Three of mine were passing for the wrong reason, and the first two are the same mistake in two places.

E2 AND E3 – AN ASSERTION SATISFIED BY A LATER FAILURE

Deleting the synthetic-address check from `resetPassword` broke nothing. With issuance already quarantined there is never a code to present, so the refusal came from "no challenge" either way — **my assertion was satisfied by a later failure, which is no assertion at all**. Same for `confirmEmailVerification`. That is the same defect the S8 finding named in the object-storage milestone, committed again by me in a new place.

The fix reaches the second layer on its own: the challenge is written one level below `issueAndSend` — the state a future code path or a hand-run migration could produce — and the reset is still refused **with a valid code in hand**. Same for verification, where accepting one would stamp `emailVerified` on a mailbox that cannot exist, and that flag is what every later account-recovery decision will lean on.

E10 – A SCOPE WITH NOTHING TO PROVE ITSELF AGAINST

Removing `providerId = 'credential'` from the reset's password write broke no test, because **every account in the suite had exactly one row in account**. With one row the scope makes no observable difference, so nothing was checking it. The new test gives the account a second provider row: unscoped, the write puts a password hash into a field an OAuth provider owns, and on an account with several rows it becomes luck rather than logic which one Better Auth then verifies against.

Two defects the screenshots found, that no test would have

- A signed-in person visiting `/sign-in` met a sign-in form with the **unverified banner above it** — two contradictions in one screen. Now redirected, on `currentActor()` rather than on a session, because a `pending_consent` account holds a session and still needs that page to render its consent step.
- **The banner rendered on the verification screen itself**, so its action linked to the page you were already reading and its "Plus tard" sat directly above a second "Plus tard" meaning something else.

In the order it blocks things.

1 Set `APP_URL` in Render, correctly, before anyone tries to sign in.

The service's own public HTTPS URL. It is `sync: false`, so Render asks for it rather than assuming — and it is not knowable until Render has created the service. Until it is right, **every email sign-in and every signup is refused** while phone sign-in keeps working, which is the most confusing possible failure. No code change can fix this from here; it is a value only you can supply.

2 Choose an email provider, and I will write the adapter.

Nothing is configured and this milestone does not pretend otherwise.

`MessageSender` is already the whole surface — no provider SDK reaches the domain, exactly as `PaymentProvider` and `ContentStorage` are handled — so an adapter is the only new code. **I did not invent a provider or an API.**

Worth comparing on deliverability to African inboxes and on price: **Resend, Postmark, Brevo, Amazon SES**. Once chosen, the variables follow the `CONTENT_S3_*` posture — `EMAIL_PROVIDER`, `EMAIL_API_KEY` (secret, `sync: false`), `EMAIL_FROM`, `EMAIL_REPLY_TO` — declared in `render.yaml`, never valued in Git.

Until then: `ConsoleSender` runs, refuses `NODE_ENV=production` without `ALLOW_CONSOLE_SENDER=yes`, and codes are read from the log. **Acceptable for a preview. Never for production.**

3 Decide what the Entrepreneur subscription costs and how it is paid.

The price is already data — an `UPDATE` on `platform_settings`, not a deploy. The payment is not: it needs `iPayMoney`, whose adapter still throws rather than pretending, and whose documentation is still unavailable. Until then a subscription can only be `pending_payment` and the screens say so in plain French.

When the provider exists, activation is one function and one test: a subscription becomes `active` **only** against a verified payment, with a period, and the guard test in section A is what will tell you if a later change smuggled in a second route.

4 Decide what happens to a phone account that wants an address.

Nothing backfills one, deliberately — that was your instruction and it is the right one. What does not exist yet is the screen where somebody chooses to add one: verify the new address, then swap it in place of the synthetic one. That is a small milestone of its own, and it is the thing that eventually lets the synthetic domain be retired.

5

Deploy and re-run the browser suite against the deployment.

Everything here ran against a local production build. The preview deployment is still blocked on the same thing the R2 milestone was: this sandbox cannot reach `onrender.com`.

NOTHING HERE NEEDS A DECISION FROM ME

Items 1 and 5 are values and access only you have. Item 2 is a commercial choice. Item 3 waits on iPayMoney. Item 4 is the next milestone whenever you want it. **The code is complete for what this milestone was asked to do**, and honestly described as: email and password work, no email is delivered, and no subscription can be paid for.

Your nineteen constraints

#	CONSTRAINT	STATUS
1	Email + password the main path for new users	DONE
2	Signup starts with the programme choice	DONE
3	Choosing Entrepreneur never means paid or active	STRUCTURAL
4	Programme / subscription / payment / activation kept apart; no second account	STRUCTURAL
5	Email verification: hashed, expiring, single use, attempt-bounded, rate-limited, no leak	DONE
6	Dashboard reachable before verification	DONE
7	Verification is not consent; consent rules unchanged	UNTOUCHED
8	Forgot password uses the authoritative challenge mechanism	DONE
9	Logout invalidates server-side	DONE
10	Existing phone accounts preserved; no backfill, no deletion, no rewritten addresses	DONE
11	Old mechanism not removed	UNTOUCHED
12	Existing <code>MessageSender</code> ; no invented provider or API	DONE
13	No simulated payment; iPayMoney out of scope	DONE
14	No referral model invented	NONE EXISTS
15	No activation bonus — no constant, table, transaction or code path	NONE EXISTS
16	Sahel design intact; only auth screens touched	DONE

#	CONSTRAINT	STATUS
17	No ledger change, no parallel balance, no wallet logic	UNTOUCHED
18	The tests planned in the audit, implemented and passing	DONE
19	This report	THIS DOCUMENT

Nothing is merged. The work is on `claude/create-app-repository-lcxsih` and waits for your word.

[↑ BACK TO THE TOP](#)